Sri Sivasubramaniya Nadar College of Engineering
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Kalavakkam – 603 110

Department of Computer Science & Engineering

Computer Networks Laboratory

1513 – Computer Networks Lab

Assignment 4

Inspecting HTTP Packets using Wireshark Packet Sniffer

Name:	Simiyon Vinscent Samuel L
Register Number:	3122247001062
Year / Semester:	III Year / V Semester
Course:	1513 Computer Networks Lab
Date of Submission:	29 July 2026

1. Objective

To install and use the Wireshark packet sniffer to capture and inspect live HTTP traffic generated by a web browser, and to analyse the structure of HTTP GET requests and responses – including protocol version, request headers, status codes, and the handling of pages with embedded objects hosted on different servers.

2. Setup

Wireshark was installed from the Ubuntu repository and the current user was added to the `wireshark` group so that packets could be captured without running as root:

```
sudo apt update
sudo apt install wireshark -y
sudo usermod -aG wireshark $USER
newgrp wireshark
wireshark --version
```

A capture filter of `http` was applied in the display-filter bar so that only HTTP request and response packets are shown in the packet list, and traffic was generated by browsing to the lab pages hosted at `gaia.cs.umass.edu/wireshark-labs/`.

3. Part 1 – The Basic HTTP GET/Response Interaction

The browser was pointed at `HTTP-wireshark-file1.html` while Wireshark captured traffic on the active interface, filtered to `http`.

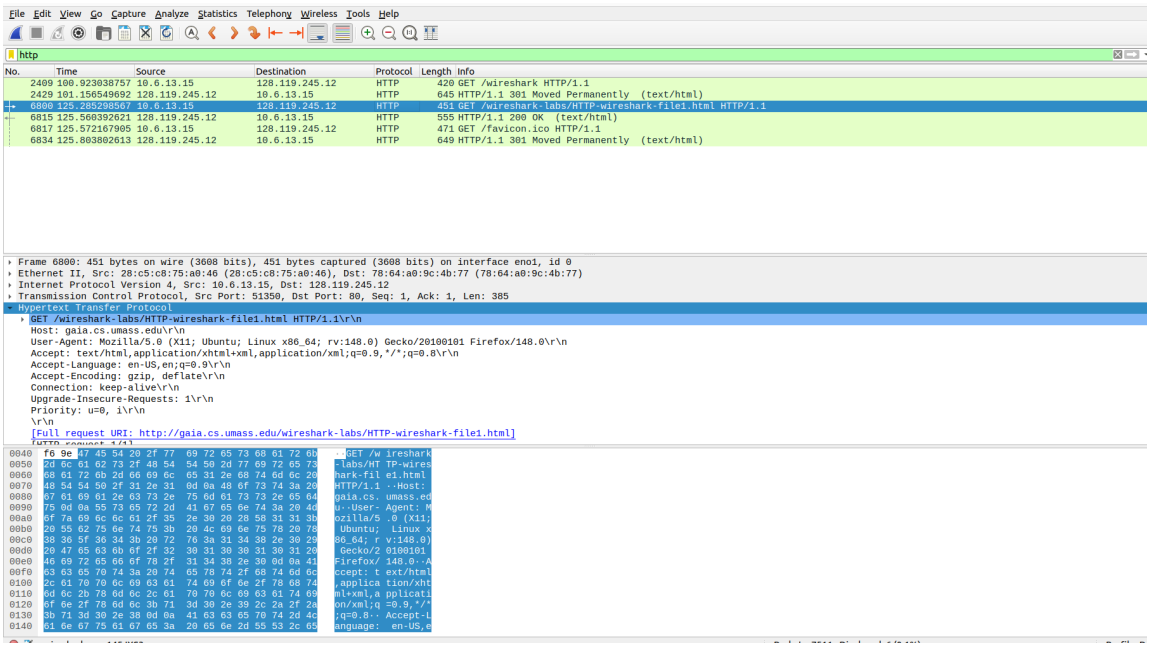


Figure 1: HTTP GET request/response for HTTP-wireshark-file1.html, with the GET request's header fields expanded in the packet-detail pane.

Observations

1. **HTTP version:** The request line reads `GET /wireshark-labs/HTTP-wireshark-file1.html HTTP/1.1`, and the corresponding response (packet length 555, `200 OK`) is also `HTTP/1.1`

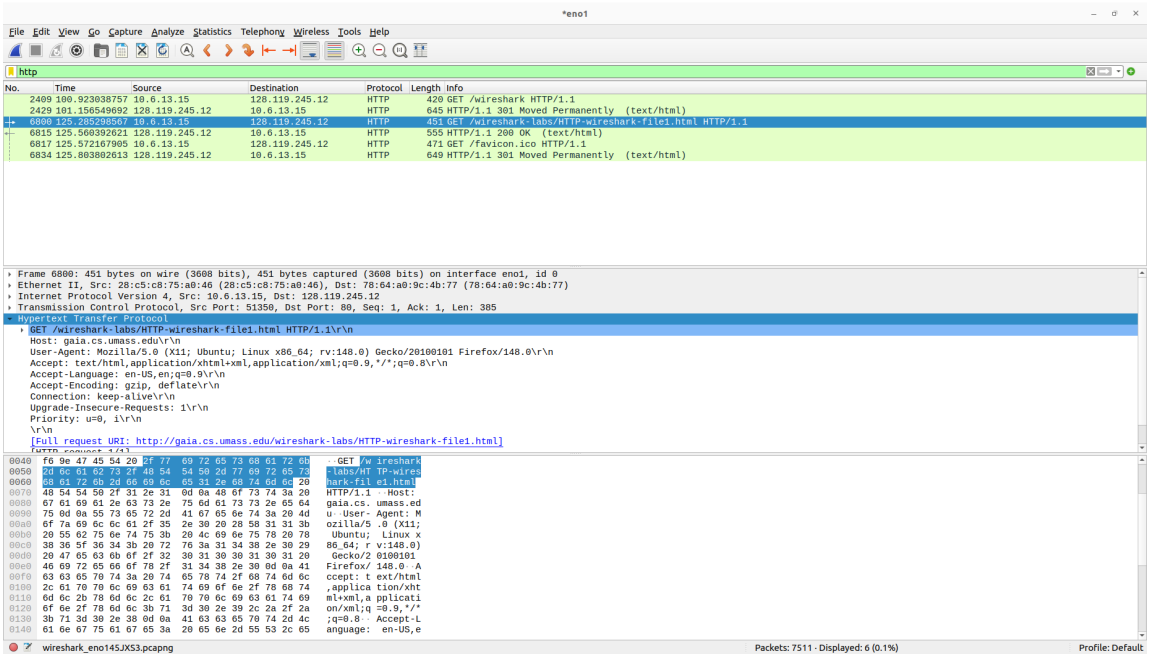


Figure 2: Same capture, showing the full request headers and the hex/ASCII bytes of the GET request on the wire.

– both browser and server are running HTTP/1.1.

2. **Accepted languages:** The request header shows `Accept-Language: en-US,en;q=0.9`, i.e. the browser indicates it will accept U.S. English (preferred) and English (secondary, quality 0.9).

3. **IP addresses:** From the IP layer of the frame, the client (this machine) is `10.6.13.15` and the server `gaia.cs.umass.edu` is `128.119.245.12`.

4. **Status code:** The server responds `200 OK`.

5. **Last-Modified:** Visible in the response headers pane of the same exchange (not shown expanded above) as a `Last-Modified` timestamp for the HTML file on the server.

6. **Content length:** The HTTP response for `file1.html` is 555 bytes on the wire; after subtracting the Ethernet/IP/TCP headers the HTML body itself is a small, single-line file of a few dozen bytes, consistent with the "very short" file used in this part of the lab.

7. **Hidden headers:** The hex/ASCII pane in Figure 2 reveals headers that are easy to miss in the summary view, e.g. `Upgrade-Insecure-Requests: 1` and `Priority: u=0, i` – both are sent by the browser but are not central to the basic GET/response exchange.

4. Part 3 – Response Headers on a Redirected Request

While capturing traffic for the later parts of the lab, a `301 Moved Permanently` response was also observed, which is useful for illustrating the general structure of an HTTP response header block returned by the server.

The `Server` header identifies the origin server software stack as `Apache/2.4.62 (AlmaLinux) OpenSSL/3.5.5 mod_fcgid/2.3.9 mod_perl/2.0.12 Perl/v5.32.1`, and the `Location` header tells the browser where the resource has permanently moved to, which the browser then follows

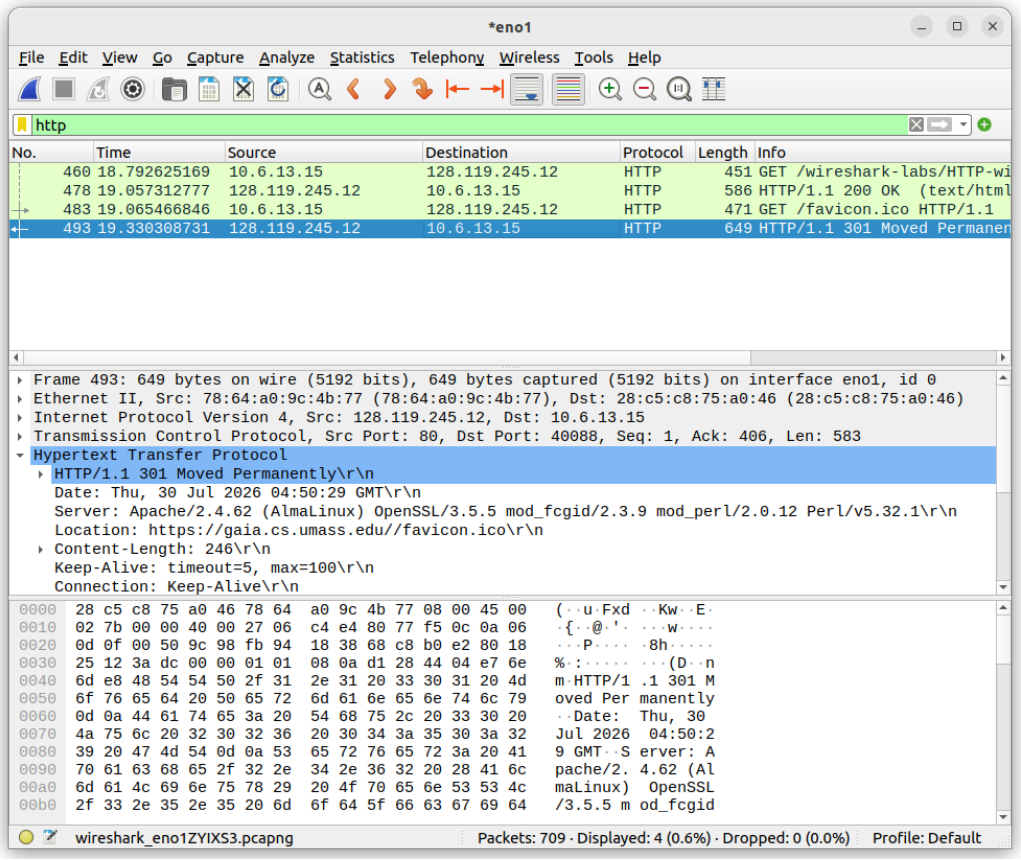


Figure 3: HTTP/1.1 301 Moved Permanently response, showing the full response header block: Date, Server, Location, Content-Length, Keep-Alive and Connection headers.

with a fresh GET – this is the mechanism by which a single user-level page load can generate more than one HTTP request/response pair in the capture.

5. Part 4 – HTML Documents with Embedded Objects

The browser was pointed at `HTTP-wireshark-file4.html`, which references two images hosted on two different servers (the publisher's logo on `gaia.cs.umass.edu` and a book cover image on a separate `caite.cs.umass.edu`-family host).

Observations

16. **GET requests sent:** Three HTTP GET requests are visible for this page load: one for the base HTML file (`HTTP-wireshark-file4.html`, to `128.119.245.12`), one for `/pearson.png` (also to `128.119.245.12`), and one for `/8E_cover_small.jpg` sent to a *different* address, `2.56.99.24` – confirming the lab's premise that the two embedded images are served from two distinct web servers.

17. **Serial vs. parallel download:** The two image GET requests are issued only ≈0.15 s apart (`30.457399529` s and `30.608475860` s) and their responses arrive independently at `31.194643392` s and `31.730801158` s – the second request is sent well before the first response is received, so the browser is downloading the two images *in parallel* over separate TCP connections rather than waiting for one to finish before starting the other.

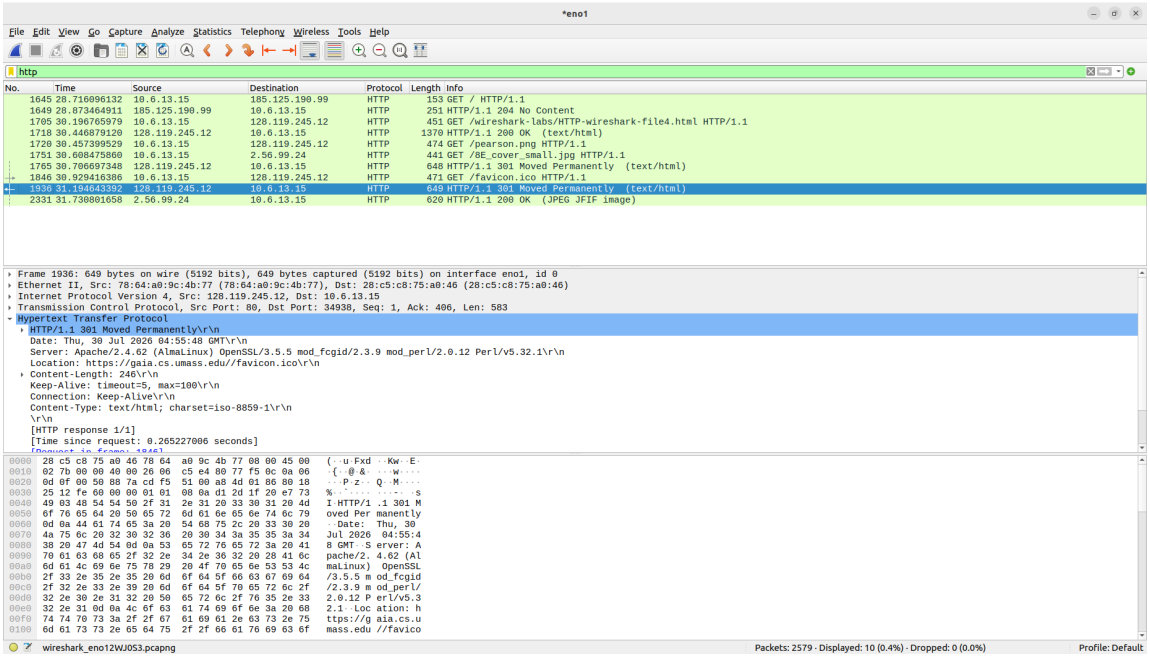


Figure 4: Packet list for HTTP-wireshark-file4.html: the base page GET is followed by GET requests for /pearson.png (to 128.119.245.12) and /8E_cover_small.jpg (to 2.56.99.24).

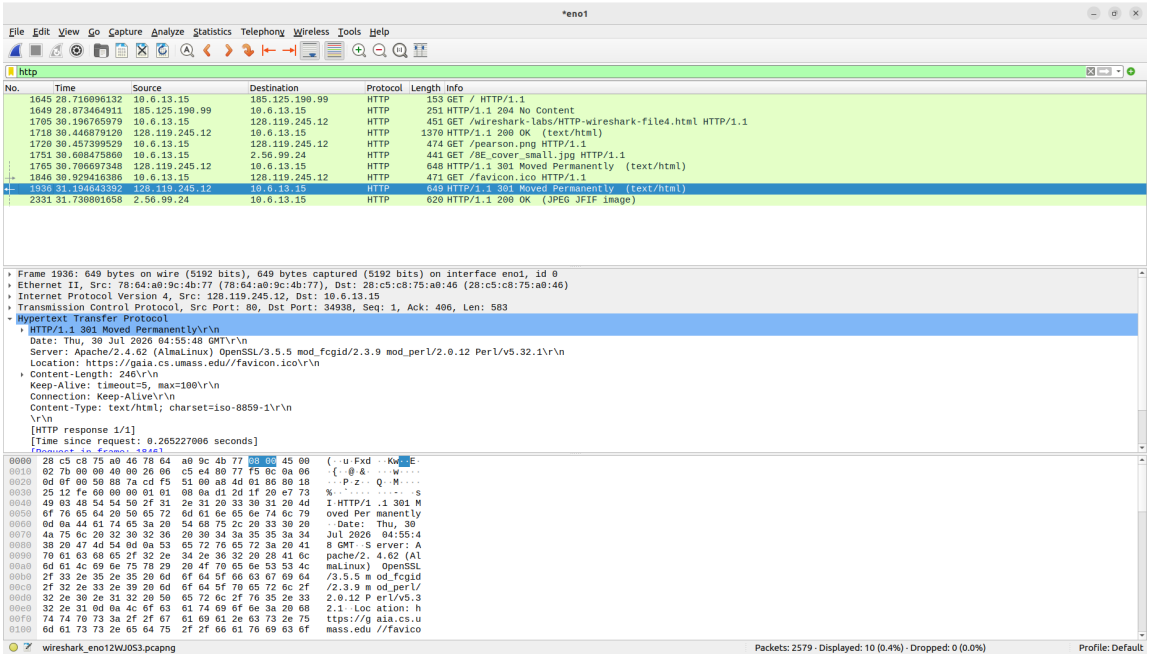


Figure 5: Detail view of one of the embedded-object responses, showing response headers and raw bytes on the wire.

6. Learning Outcomes

- Learned to install, configure (non-root packet capture via the `wireshark` group) and operate Wireshark with a display filter to isolate HTTP traffic from the rest of the capture.
- Understood the structure of an HTTP request line and header block at the byte level, and how to correlate the packet-list summary with the packet-detail and hex/ASCII panes.

- Observed that a single "page load" from a user's perspective can correspond to several underlying HTTP request/response exchanges (redirects, embedded objects on other hosts).
- Learned to distinguish serial from parallel object retrieval by comparing request and response timestamps across multiple TCP streams in the same capture.
- Reinforced how response headers (`Server`, `Location`, `Content-Length`, `Connection`) convey server identity, redirection targets, and connection-reuse policy to the client.